

I. SYSTEM SETUP

This section outlines an overview of the system, hardware specifications, simulation parameters, our energy model, latency model, workload model, EPLB algorithm, and system operation.

A. System Overview

A Python-based discrete-event simulation framework was used to evaluate energy-aware load balancing for elastic AI inference workloads. The system was designed as a three-tier architecture: client applications, an EPLB load balancer, and heterogeneous inference node pools. Client applications submit inference requests for AI model execution, such as text, images, audio, video, and binary data, and receive prediction results. The EPLB load balancer is the central routing component responsible for making energy-aware decisions and routing the requests across available resources. Inference node pools consist of heterogeneous computing instances with varying power consumption profiles and performance characteristics, including general-purpose CPU instances and GPU-accelerated instances with different capabilities.

B. Hardware and instance specifications

The simulation uses 2023-2024 generation hardware models: CPUs, mid-range GPUs, and high-performance GPUs. These tiers were chosen based on the tradeoff between latency, capacity, and energy consumption. The instance pool configuration defines how many units of each hardware type are provisioned in the simulated cloud cluster, establishing the overall system capacity. Table 1 shows the hardware specifications, and Table 2 shows the workload specifications used in this study. This setup was conducted to evaluate how latency evolves under varying loads and how energy consumption scales with utilization.

Instance type	Idle power	Active power	Base inference time	Max RPS
AWS m7i.xlarge	25 W	75 W	95 ms	50
NVIDIA L4	72 W	72 W	4 ms	150
NVIDIA H100	150 W	700 W	1.5 ms	300

TABLE I
HARDWARE SPECIFICATION PER INSTANCE TYPE

Each hardware type in Table 1 is described using several key parameters that characterize its performance, latency, and energy behavior. Idle power in Watts: The amount of power (in watts) that the device consumes when it is powered on but not actively processing any requests. This value represents the base power consumption when the system is idle.

Active power in Watts: The power draw (in watts) when the device is operating under full load. This value reflects peak power consumption during active use.

Base inference time in ms: The average time (in milliseconds) required for a single inference request when the device is lightly loaded. It serves as the baseline latency before additional delays are introduced by queuing or high utilization.

Max Capacity in requests per second (rps): The maximum number of inference requests that the device can process per second under optimal conditions. This represents the throughput limit of that hardware type.

Instance Type	Count	Total capacity ($Count \times MaxRPS$)
AWS m7i.xlarge	20	1000
NVIDIA L4	10	1500
NVIDIA H100	5	1500

TABLE II
INSTANCE POOL USED IN THE STUDY

The instance pool in Table 2 defines how many instances of each hardware type are provisioned in the simulated cloud cluster. Here, the count represents the number of instances used in the study, and the total capacity of each instance type is the number of instances multiplied by the maximum RPS that it can support. Each instance contributes to the total system throughput according to its maximum capacity, RPS, resulting in a combined capacity of 4,000 requests per second.

C. Simulation Parameters

The simulation configuration defines the key parameters that are used to govern how the system operates and evaluates performance. The setup runs for 24 hours with a 1-minute time resolution, and a default latency threshold of 100 milliseconds is used as a filter to ensure that requests are assigned only to hardware capable of meeting acceptable response times. The

simulation compares multiple load balancing algorithms: Energy-Proportional Load Balancing (EPLB), Round Robin, and Least Connections, under two distinct workload patterns, Sinusoidal (smooth, periodic load variation) and Spiky(sudden, burst traffic) [1].

- **Simulation duration:** `simulation_hours = 24`
- **Time resolution:** `time_resolution_minutes = 1` (per-minute stepping)
- **Latency filter:** `l_max_default = 100 ms`
- **Algorithms compared:**
`{EPLB, RoundRobin, LeastConnections, Thermal Aware and Path Based}`
- **Workload patterns:** `{Sinusoidal, Spiky}`

D. Workload Model

Two functions (`generate_sinusoidal_workload` and `generate_spiky_workload`) were defined to produce per-minute RPS values for 24 hours with 1440 samples. The sinusoidal workload represents predictable, time-varying traffic, such as user activity following a diurnal cycle.

$$\text{Base_RPS} = 1000 + 800 \sin(2\pi \times \text{hour}/24) \quad (1)$$

Its base rate is defined as in Equation 1, producing values between approximately 200 and 1,800 requests per second, with added uniform noise ($\pm 10\%$) to introduce natural variability, as shown in Figure 1.

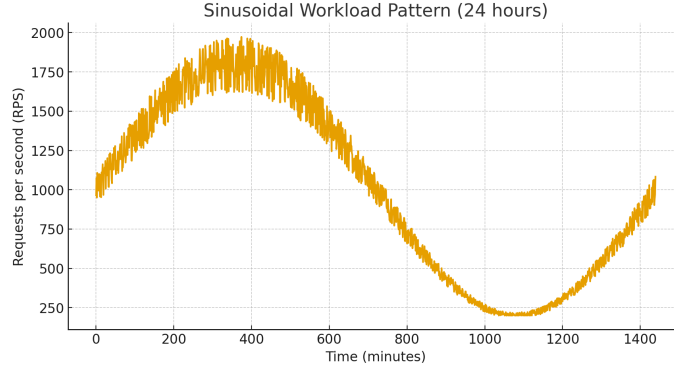


Fig. 1. Sinusoidal Workload Pattern

In contrast, the spiky workload captures the unpredictable, burst behavior often seen in real-time or event-driven systems. It maintains a baseline of around 450–550 requests per second, with random bursts (occurring in about 8–12% of time steps) reaching intensities of 2,800–3,200 requests per second, as shown in Figure 2.

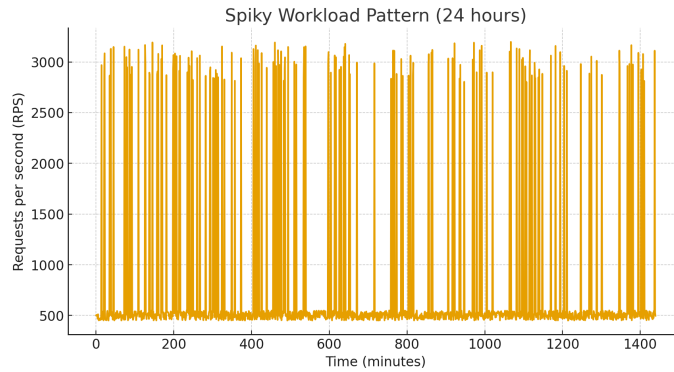


Fig. 2. Spiky or Burst Workload Pattern

E. Energy Model

We compute total energy (kWh) per instance type as the sum of active and idle energy over the simulation:

$$\text{Total processing time of all the requests in hours (t)} = \frac{\text{number of requests} \times \text{Time taken per request in ms (tp)}}{1000 \times 60 \times 60} \quad (2)$$

$$\text{Active energy (kWh)} = \frac{\text{Active power (W)} \times t}{1000} \quad (3)$$

$$\text{Idle energy when system is inactive (kWh)} = \frac{\text{Idle power (W)} \times \text{Idle instances} \times \text{Idle time (hours)}}{1000} \quad (4)$$

$$\text{Total energy (kWh)} = \text{Active energy} + \text{Idle energy} \quad (5)$$

Example: for a CPU with an active power of 75 W and a base inference time of 95 ms, the energy per inference is

$$\frac{75 \times 95}{3,600,000} \approx 1.979 \times 10^{-3} \text{ Wh.}$$

Thus 1,000 requests require about 1.979 Wh (0.001979 kWh). If many instances remain idle for 24 h, idle energy can dominate total consumption and so must be accounted for explicitly.

F. Latency Model

The proposed dynamic latency model introduces a nonlinear mapping between system utilization and response latency. The function `calculate_dynamic_latency` receives an instance type and its current utilization (0-100%) and outputs the adjusted latency in milliseconds.

Utilization is first converted from a percentage to a fraction. The model was designed to calculate the piecewise delay factor to the instance's base inference latency (`base_inference_time_ms`) as defined in the hardware specifications:

$$\text{delay_factor} = \begin{cases} 1.0, & \text{if util} < 0.5 \\ 1.0 + (\text{util} - 0.5) \times 2, & 0.5 \leq \text{util} < 0.8 \\ 1.6 + (\text{util} - 0.8) \times 10, & 0.8 \leq \text{util} < 0.95 \\ 3.1 + (\text{util} - 0.95) \times 40, & \text{util} \geq 0.95 \end{cases}$$

The adjusted latency is then

$$\text{latency}_{\text{ms}} = \text{base_latency}_{\text{ms}} \times \text{delay_factor}.$$

For example, for a CPU with a base latency of 95 ms:

- $\text{util} = 0.7 \Rightarrow \text{factor} = 1.4 \Rightarrow \text{latency} \approx 133 \text{ ms}$
- $\text{util} = 0.9 \Rightarrow \text{factor} = 2.6 \Rightarrow \text{latency} \approx 247 \text{ ms}$
- $\text{util} = 0.99 \Rightarrow \text{factor} = 4.7 \Rightarrow \text{latency} \approx 447 \text{ ms}$

This empirical queuing-style model ensures that latency remains stable under light load but rises sharply near capacity, effectively simulating congestion and enabling a fair comparison of load-balancing algorithms under varying utilization.